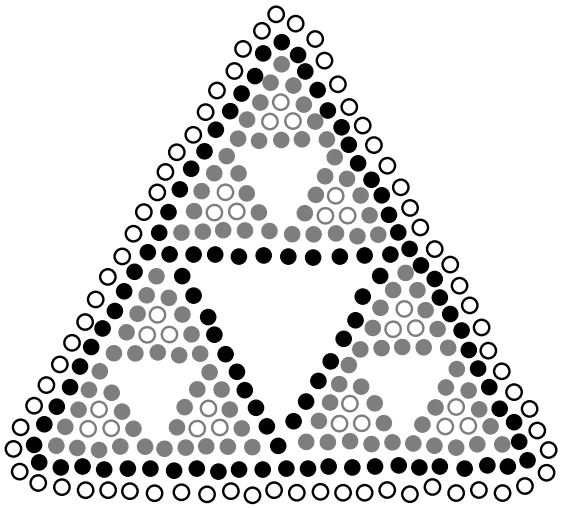
Chapter 4 Recursion



Contents

4.1 Illustrative Examples 150
4.1.1 The Factorial Function 150
4.1.2 Drawing an English Ruler 152
4.1.3 Binary Search 155
4.1.4 File Systems 157
4.2 Analyzing Recursive Algorithms 161
4.3 Recursion Run Amok 165
4.3.1 Maximum Recursive Depth in Python 168
4.4 Further Examples of Recursion 169
4.4.1 Linear Recursion 169
4.4.2 Binary Recursion 174
4.4.3 Multiple Recursion 175
4.5 Designing Recursive Algorithms 177
4.6 Eliminating Tail Recursion 178
4.7 Exercises 180

One way to describe repetition within a computer program is the use of loops, such as Python's while-loop and for-loop constructs described in Section 1.4.2. An entirely different way to achieve repetition is through a process known as ***recursion***.

Recursion is a technique by which a function makes one or more calls to itself during execution, or by which a data structure relies upon smaller instances of the very same type of structure in its representation. There are many examples of recursion in art and nature. For example, fractal patterns are naturally recursive. A physical example of recursion used in art is in the Russian Matryoshka dolls. Each doll is either made of solid wood, or is hollow and contains another Matryoshka doll inside it.

In computing, recursion provides an elegant and powerful alternative for performing repetitive tasks. In fact, a few programming languages (e.g., Scheme, Smalltalk) do not explicitly support looping constructs and instead rely directly on recursion to express repetition. Most modern programming languages support functional recursion using the identical mechanism that is used to support traditional forms of function calls. When one invocation of the function make a recursive call, that invocation is suspended until the recursive call completes.

Recursion is an important technique in the study of data structures and algorithms. We will use it prominently in several later chapters of this book (most notably, Chapters 8 and 12). In this chapter, we begin with the following four illustrative examples of the use of recursion, providing a Python implementation for each.

- The ***factorial function*** (commonly denoted as $n!$) is a classic mathematical function that has a natural recursive definition.
- An ***English ruler*** has a recursive pattern that is a simple example of a fractal structure.
- ***Binary search*** is among the most important computer algorithms. It allows us to efficiently locate a desired value in a data set with upwards of billions of entries.
- The ***file system*** for a computer has a recursive structure in which directories can be nested arbitrarily deeply within other directories. Recursive algorithms are widely used to explore and manage these file systems.

We then describe how to perform a formal analysis of the running time of a recursive algorithm and we discuss some potential pitfalls when defining recursions. In the balance of the chapter, we provide many more examples of recursive algorithms, organized to highlight some common forms of design.

4.1 Illustrative Examples

4.1.1 The Factorial Function

To demonstrate the mechanics of recursion, we begin with a simple mathematical example of computing the value of the ***factorial function***. The factorial of a positive integer n, denoted $n!$, is defined as the product of the integers from 1 to n. If $n = 0$, then $n!$ is defined as 1 by convention. More formally, for any integer $n \geq 0$,

$$n! = \begin{cases} 1 & \text{if } n = 0 \\ n \cdot (n-1) \cdot (n-2) \cdots 3 \cdot 2 \cdot 1 & \text{if } n \geq 1. \end{cases}$$

For example, $5! = 5 \cdot 4 \cdot 3 \cdot 2 \cdot 1 = 120$. The factorial function is important because it is known to equal the number of ways in which n distinct items can be arranged into a sequence, that is, the number of ***permutations*** of n items. For example, the three characters `a`, `b`, and `c` can be arranged in $3! = 3 \cdot 2 \cdot 1 = 6$ ways: `abc`, `acb`, `bac`, `bca`, `cab`, and `cba`.

There is a natural recursive definition for the factorial function. To see this, observe that $5! = 5 \cdot (4 \cdot 3 \cdot 2 \cdot 1) = 5 \cdot 4!$. More generally, for a positive integer n, we can define $n!$ to be $n \cdot (n-1)!$. This ***recursive definition*** can be formalized as

$$n! = \begin{cases} 1 & \text{if } n = 0 \\ n \cdot (n-1)! & \text{if } n \geq 1. \end{cases}$$

This definition is typical of many recursive definitions. First, it contains one or more ***base cases***, which are defined nonrecursively in terms of fixed quantities. In this case, $n = 0$ is the base case. It also contains one or more ***recursive cases***, which are defined by appealing to the definition of the function being defined.

A Recursive Implementation of the Factorial Function

Recursion is not just a mathematical notation; we can use recursion to design a Python implementation of a factorial function, as shown in Code Fragment 4.1.

```
1 def factorial(n):
2   if n == 0:
3     return 1
4   else:
5     return n * factorial(n-1)
```

Code Fragment 4.1: A recursive implementation of the factorial function.

This function does not use any explicit loops. Repetition is provided by the repeated recursive invocations of the function. There is no circularity in this definition, because each time the function is invoked, its argument is smaller by one, and when a base case is reached, no further recursive calls are made.

We illustrate the execution of a recursive function using a ***recursion trace***. Each entry of the trace corresponds to a recursive call. Each new recursive function call is indicated by a downward arrow to a new invocation. When the function returns, an arrow showing this return is drawn and the return value may be indicated alongside this arrow. An example of such a trace for the factorial function is shown in Figure 4.1.

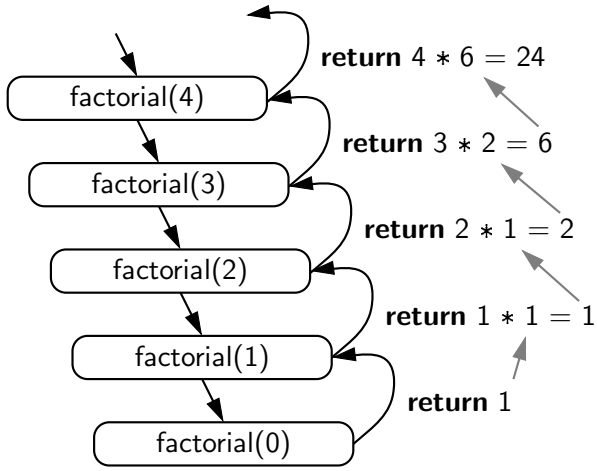


Figure 4.1: A recursion trace for the call factorial(5).

A recursion trace closely mirrors the programming language's execution of the recursion. In Python, each time a function (recursive or otherwise) is called, a structure known as an ***activation record*** or ***frame*** is created to store information about the progress of that invocation of the function. This activation record includes a namespace for storing the function call's parameters and local variables (see Section 1.10 for a discussion of namespaces), and information about which command in the body of the function is currently executing.

When the execution of a function leads to a nested function call, the execution of the former call is suspended and its activation record stores the place in the source code at which the flow of control should continue upon return of the nested call. This process is used both in the standard case of one function calling a different function, or in the recursive case in which a function invokes itself. The key point is that there is a different activation record for each active call.

4.1.2 Drawing an English Ruler

In the case of computing a factorial, there is no compelling reason for preferring recursion over a direct iteration with a loop. As a more complex example of the use of recursion, consider how to draw the markings of a typical English ruler. For each inch, we place a tick with a numeric label. We denote the length of the tick designating a whole inch as the ***major tick length***. Between the marks for whole inches, the ruler contains a series of ***minor ticks***, placed at intervals of $1/2$ inch, $1/4$ inch, and so on. As the size of the interval decreases by half, the tick length decreases by one. Figure 4.2 demonstrates several such rulers with varying major tick lengths (although not drawn to scale).

```
---- 0
-
--
-
---
-
--
-
---- 1
-
--
-
---
-
--
-
---- 2
(a)
```

```
----- 0
-
--
-
---
-
--
-
----
-
--
-
---
-
--
-
----- 1
(b)
```

```
--- 0
-
--
-
--- 1
-
--
-
--- 2
-
--
-
--- 3
(c)
```

Figure 4.2: Three sample outputs of an English ruler drawing: (a) a 2-inch ruler with major tick length 4; (b) a 1-inch ruler with major tick length 5; (c) a 3-inch ruler with major tick length 3.

A Recursive Approach to Ruler Drawing

The English ruler pattern is a simple example of a ***fractal***, that is, a shape that has a self-recursive structure at various levels of magnification. Consider the rule with major tick length 5 shown in Figure 4.2(b). Ignoring the lines containing 0 and 1, let us consider how to draw the sequence of ticks lying between these lines. The central tick (at 1/2 inch) has length 4. Observe that the two patterns of ticks above and below this central tick are identical, and each has a central tick of length 3.

In general, an interval with a central tick length $L \geq 1$ is composed of:

- An interval with a central tick length $L-1$
- A single tick of length L
- An interval with a central tick length $L-1$

Although it is possible to draw such a ruler using an iterative process (see Exercise P-4.25), the task is considerably easier to accomplish with recursion. Our implementation consists of three functions, as shown in Code Fragment 4.2. The main function, draw_ruler, manages the construction of the entire ruler. Its arguments specify the total number of inches in the ruler and the major tick length. The utility function, draw_line, draws a single tick with a specified number of dashes (and an optional string label, that is printed after the tick).

The interesting work is done by the recursive draw_interval function. This function draws the sequence of minor ticks within some interval, based upon the length of the interval's central tick. We rely on the intuition shown at the top of this page, and with a base case when $L = 0$ that draws nothing. For $L \geq 1$, the first and last steps are performed by recursively calling draw_interval$(L-1)$. The middle step is performed by calling the function draw_line(L).

```
1  def draw_line(tick_length, tick_label=''):
2    """Draw one line with given tick length (followed by optional label)."""
3    line = '-' * tick_length
4    if tick_label:
5      line += ' ' + tick_label
6    print(line)
7
8  def draw_interval(center_length):
9    """Draw tick interval based upon a central tick length."""
10   if center_length > 0:                    # stop when length drops to 0
11     draw_interval(center_length - 1)       # recursively draw top ticks
12     draw_line(center_length)               # draw center tick
13     draw_interval(center_length - 1)       # recursively draw bottom ticks
14
15 def draw_ruler(num_inches, major_length):
16   """Draw English ruler with given number of inches, major tick length."""
17   draw_line(major_length, '0')             # draw inch 0 line
18   for j in range(1, 1 + num_inches):
19     draw_interval(major_length - 1)        # draw interior ticks for inch
20     draw_line(major_length, str(j))        # draw inch j line and label
```

Code Fragment 4.2: A recursive implementation of a function that draws a ruler.

Illustrating Ruler Drawing Using a Recursion Trace

The execution of the recursive draw_interval function can be visualized using a recursion trace. The trace for draw_interval is more complicated than in the factorial example, however, because each instance makes two recursive calls. To illustrate this, we will show the recursion trace in a form that is reminiscent of an outline for a document. See Figure 4.3.

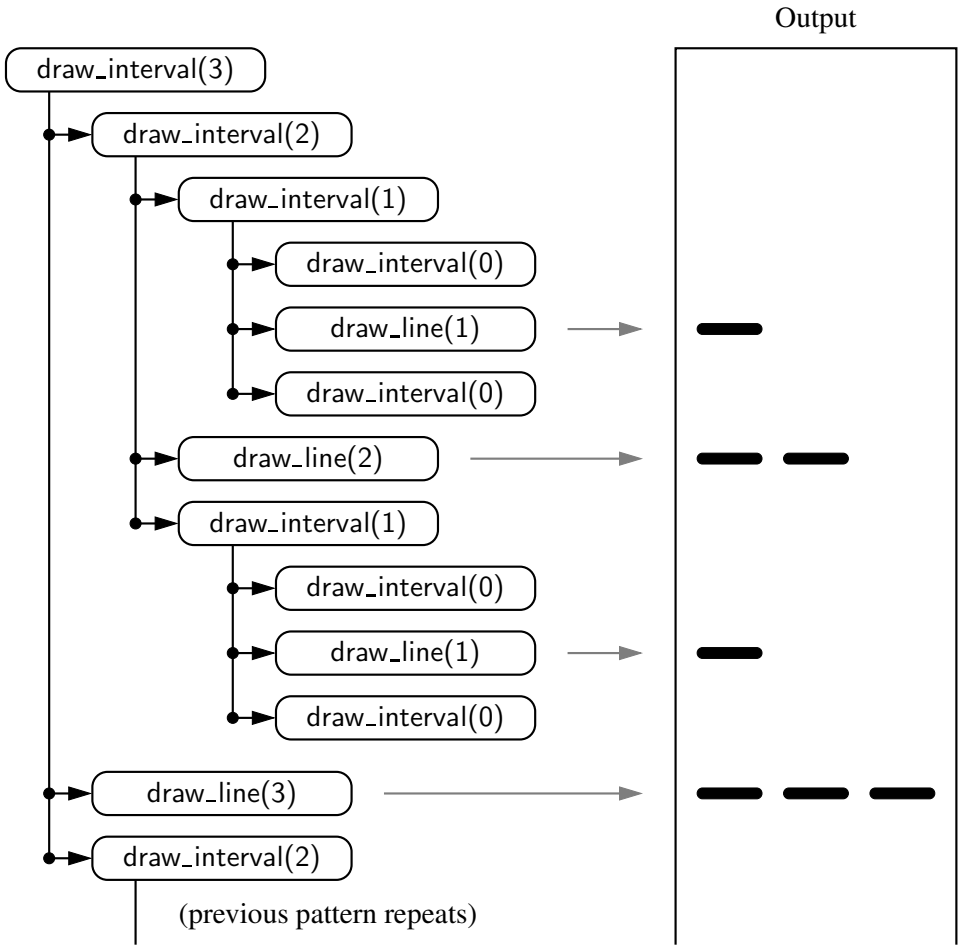


Figure 4.3: A partial recursion trace for the call draw_interval(3). The second pattern of calls for draw_interval(2) is not shown, but it is identical to the first.

4.1.3 Binary Search

In this section, we describe a classic recursive algorithm, ***binary search***, that is used to efficiently locate a target value within a sorted sequence of n elements. This is among the most important of computer algorithms, and it is the reason that we so often store data in sorted order (as in Figure 4.4).

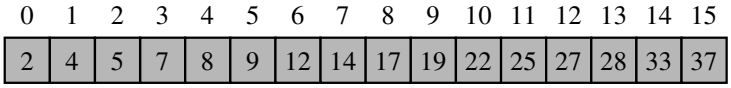


Figure 4.4: Values stored in sorted order within an indexable sequence, such as a Python list. The numbers at top are the indices.

When the sequence is ***unsorted***, the standard approach to search for a target value is to use a loop to examine every element, until either finding the target or exhausting the data set. This is known as the ***sequential search*** algorithm. This algorithm runs in $O(n)$ time (i.e., linear time) since every element is inspected in the worst case.

When the sequence is ***sorted*** and ***indexable***, there is a much more efficient algorithm. (For intuition, think about how you would accomplish this task by hand!) For any index j, we know that all the values stored at indices $0,\ldots,j-1$ are less than or equal to the value at index j, and all the values stored at indices $j+1,\ldots,n-1$ are greater than or equal to that at index j. This observation allows us to quickly "home in" on a search target using a variant of the children's game "high-low." We call an element of the sequence a ***candidate*** if, at the current stage of the search, we cannot rule out that this item matches the target. The algorithm maintains two parameters, low and high, such that all the candidate entries have index at least low and at most high. Initially, $\mathsf{low} = 0$ and $\mathsf{high} = n-1$. We then compare the target value to the median candidate, that is, the item data[mid] with index

$$\mathsf{mid} = \lfloor (\mathsf{low} + \mathsf{high})/2 \rfloor .$$

We consider three cases:

- If the target equals data[mid], then we have found the item we are looking for, and the search terminates successfully.
- If target $<$ data[mid], then we recur on the first half of the sequence, that is, on the interval of indices from low to $\mathsf{mid}-1$.
- If target $>$ data[mid], then we recur on the second half of the sequence, that is, on the interval of indices from $\mathsf{mid}+1$ to high.

An unsuccessful search occurs if low $>$ high, as the interval $[\mathsf{low}, \mathsf{high}]$ is empty.

This algorithm is known as ***binary search***. We give a Python implementation in Code Fragment 4.3, and an illustration of the execution of the algorithm in Figure 4.5. Whereas sequential search runs in $O(n)$ time, the more efficient binary search runs in $O(\log n)$ time. This is a significant improvement, given that if n is one billion, $\log n$ is only 30. (We defer our formal analysis of binary search's running time to Proposition 4.2 in Section 4.2.)

```
1  def binary_search(data, target, low, high):
2    """Return True if target is found in indicated portion of a Python list.
3
4    The search only considers the portion from data[low] to data[high] inclusive.
5    """
6    if low > high:
7      return False                          # interval is empty; no match
8    else:
9      mid = (low + high) // 2
10     if target == data[mid]:               # found a match
11       return True
12     elif target < data[mid]:
13       # recur on the portion left of the middle
14       return binary_search(data, target, low, mid - 1)
15     else:
16       # recur on the portion right of the middle
17       return binary_search(data, target, mid + 1, high)
```

Code Fragment 4.3: An implementation of the binary search algorithm.

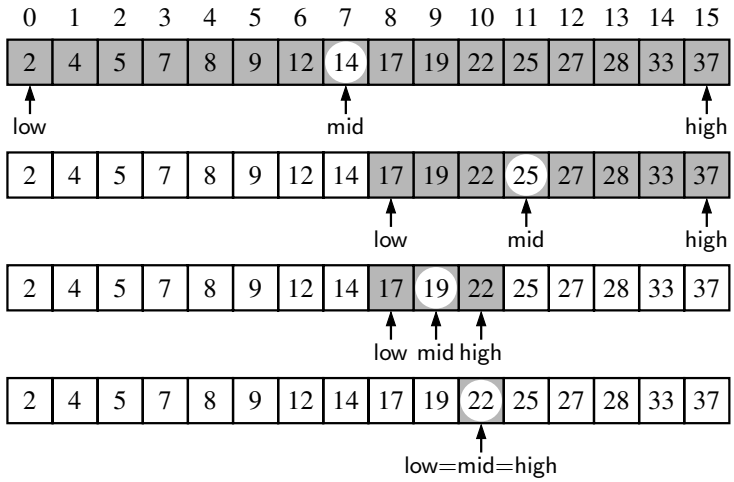


Figure 4.5: Example of a binary search for target value 22.

4.1.4 File Systems

Modern operating systems define file-system directories (which are also sometimes called "folders") in a recursive way. Namely, a file system consists of a top-level directory, and the contents of this directory consists of files and other directories, which in turn can contain files and other directories, and so on. The operating system allows directories to be nested arbitrarily deep (as long as there is enough space in memory), although there must necessarily be some base directories that contain only files, not further subdirectories. A representation of a portion of such a file system is given in Figure 4.6.

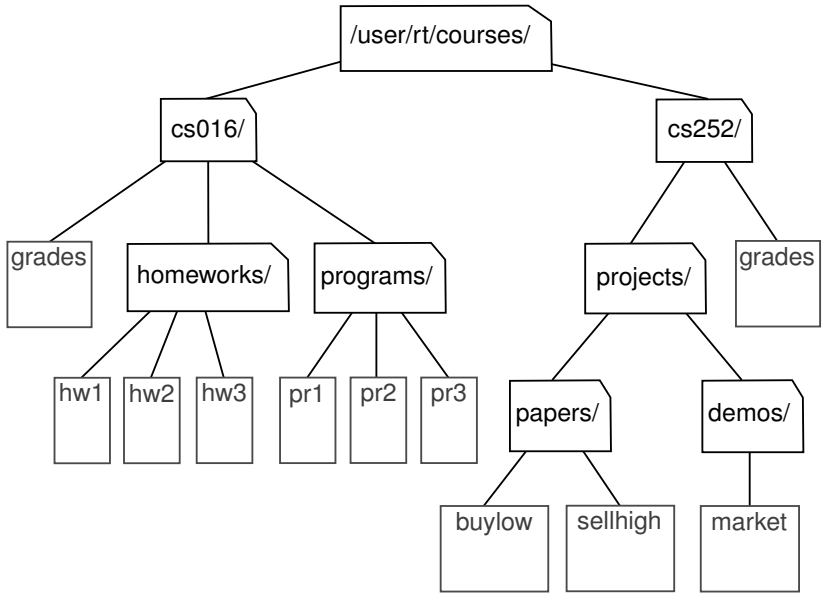


Figure 4.6: A portion of a file system demonstrating a nested organization.

Given the recursive nature of the file-system representation, it should not come as a surprise that many common behaviors of an operating system, such as copying a directory or deleting a directory, are implemented with recursive algorithms. In this section, we consider one such algorithm: computing the total disk usage for all files and directories nested within a particular directory.

For illustration, Figure 4.7 portrays the disk space being used by all entries in our sample file system. We differentiate between the *immediate* disk space used by each entry and the *cumulative* disk space used by that entry and all nested features. For example, the `cs016` directory uses only 2K of immediate space, but a total of 249K of cumulative space.

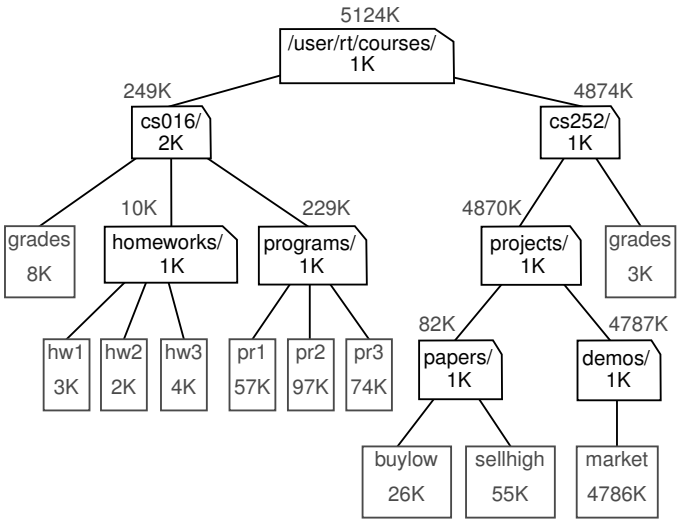


Figure 4.7: The same portion of a file system given in Figure 4.6, but with additional annotations to describe the amount of disk space that is used. Within the icon for each file or directory is the amount of space directly used by that artifact. Above the icon for each directory is an indication of the *cumulative* disk space used by that directory and all its (recursive) contents.

The cumulative disk space for an entry can be computed with a simple recursive algorithm. It is equal to the immediate disk space used by the entry plus the sum of the *cumulative* disk space usage of any entries that are stored directly within the entry. For example, the cumulative disk space for `cs016` is 249K because it uses 2K itself, 8K cumulatively in `grades`, 10K cumulatively in `homeworks`, and 229K cumulatively in `programs`. Pseudo-code for this algorithm is given in Code Fragment 4.4.

Algorithm DiskUsage(path):
Input: A string designating a path to a file-system entry
Output: The cumulative disk space used by that entry and any nested entries

```
total = size(path)                          {immediate disk space used by the entry}
if path represents a directory then
  for each child entry stored within directory path do
    total = total + DiskUsage(child)        {recursive call}
return total
```

Code Fragment 4.4: An algorithm for computing the cumulative disk space usage nested at a file-system entry. Function size returns the immediate disk space of an entry.

Python's os Module

To provide a Python implementation of a recursive algorithm for computing disk usage, we rely on Python's os module, which provides robust tools for interacting with the operating system during the execution of a program. This is an extensive library, but we will only need the following four functions:

- **os.path.getsize(path)**
  Return the immediate disk usage (measured in bytes) for the file or directory that is identified by the string path (e.g., `/user/rt/courses`).
- **os.path.isdir(path)**
  Return True if entry designated by string path is a directory; False otherwise.
- **os.listdir(path)**
  Return a list of strings that are the names of all entries within a directory designated by string path. In our sample file system, if the parameter is `/user/rt/courses`, this returns the list `['cs016', 'cs252']`.
- **os.path.join(path, filename)**
  Compose the path string and filename string using an appropriate operating system separator between the two (e.g., the / character for a Unix/Linux system, and the \ character for Windows). Return the string that represents the full path to the file.

Python Implementation

With use of the os module, we now convert the algorithm from Code Fragment 4.4 into the Python implementation of Code Fragment 4.5.

```
1  import os
2
3  def disk_usage(path):
4    """Return the number of bytes used by a file/folder and any descendents."""
5    total = os.path.getsize(path)                # account for direct usage
6    if os.path.isdir(path):                      # if this is a directory,
7      for filename in os.listdir(path):          # then for each child:
8        childpath = os.path.join(path, filename) # compose full path to child
9        total += disk_usage(childpath)           # add child's usage to total
10
11   print ('{0:<7}'.format(total), path)         # descriptive output (optional)
12   return total                                 # return the grand total
```

Code Fragment 4.5: A recursive function for reporting disk usage of a file system.

Recursion Trace

To produce a different form of a recursion trace, we have included an extraneous print statement within our Python implementation (line 11 of Code Fragment 4.5). The precise format of that output intentionally mirrors output that is produced by a classic Unix/Linux utility named du (for "disk usage"). It reports the amount of disk space used by a directory and all contents nested within, and can produce a verbose report, as given in Figure 4.8.

Our implementation of the disk_usage function produces an identical result, when executed on the sample file system portrayed in Figure 4.7. During the execution of the algorithm, exactly one recursive call is made for each entry in the portion of the file system that is considered. Because the print statement is made just before returning from a recursive call, the output shown in Figure 4.8 reflects the order in which the recursive calls are completed. In particular, we begin and end a recursive call for each entry that is nested below another entry, computing the nested cumulative disk space before we can compute and report the cumulative disk space for the containing entry. For example, we do not know the cumulative total for entry /user/rt/courses/cs016 until after the recursive calls regarding contained entries grades, homeworks, and programs complete.

```
8       /user/rt/courses/cs016/grades
3       /user/rt/courses/cs016/homeworks/hw1
2       /user/rt/courses/cs016/homeworks/hw2
4       /user/rt/courses/cs016/homeworks/hw3
10      /user/rt/courses/cs016/homeworks
57      /user/rt/courses/cs016/programs/pr1
97      /user/rt/courses/cs016/programs/pr2
74      /user/rt/courses/cs016/programs/pr3
229     /user/rt/courses/cs016/programs
249     /user/rt/courses/cs016
26      /user/rt/courses/cs252/projects/papers/buylow
55      /user/rt/courses/cs252/projects/papers/sellhigh
82      /user/rt/courses/cs252/projects/papers
4786    /user/rt/courses/cs252/projects/demos/market
4787    /user/rt/courses/cs252/projects/demos
4870    /user/rt/courses/cs252/projects
3       /user/rt/courses/cs252/grades
4874    /user/rt/courses/cs252
5124    /user/rt/courses/
```

Figure 4.8: A report of the disk usage for the file system shown in Figure 4.7, as generated by the Unix/Linux utility du (with command-line options -ak), or equivalently by our disk_usage function from Code Fragment 4.5.

4.2 Analyzing Recursive Algorithms

In Chapter 3, we introduced mathematical techniques for analyzing the efficiency of an algorithm, based upon an estimate of the number of primitive operations that are executed by the algorithm. We use notations such as big-Oh to summarize the relationship between the number of operations and the input size for a problem. In this section, we demonstrate how to perform this type of running-time analysis to recursive algorithms.

With a recursive algorithm, we will account for each operation that is performed based upon the particular ***activation*** of the function that manages the flow of control at the time it is executed. Stated another way, for each invocation of the function, we only account for the number of operations that are performed within the body of that activation. We can then account for the overall number of operations that are executed as part of the recursive algorithm by taking the sum, over all activations, of the number of operations that take place during each individual activation. (As an aside, this is also the way we analyze a nonrecursive function that calls other functions from within its body.)

To demonstrate this style of analysis, we revisit the four recursive algorithms presented in Sections 4.1.1 through 4.1.4: factorial computation, drawing an English ruler, binary search, and computation of the cumulative size of a file system. In general, we may rely on the intuition afforded by a ***recursion trace*** in recognizing how many recursive activations occur, and how the parameterization of each activation can be used to estimate the number of primitive operations that occur within the body of that activation. However, each of these recursive algorithms has a unique structure and form.

Computing Factorials

It is relatively easy to analyze the efficiency of our function for computing factorials, as described in Section 4.1.1. A sample recursion trace for our factorial function was given in Figure 4.1. To compute factorial(n), we see that there are a total of $n+1$ activations, as the parameter decreases from n in the first call, to $n-1$ in the second call, and so on, until reaching the base case with parameter 0.

It is also clear, given an examination of the function body in Code Fragment 4.1, that each individual activation of factorial executes a constant number of operations. Therefore, we conclude that the overall number of operations for computing factorial(n) is $O(n)$, as there are $n+1$ activations, each of which accounts for $O(1)$ operations.

Drawing an English Ruler

In analyzing the English ruler application from Section 4.1.2, we consider the fundamental question of how many total lines of output are generated by an initial call to draw_interval(c), where c denotes the center length. This is a reasonable benchmark for the overall efficiency of the algorithm as each line of output is based upon a call to the draw_line utility, and each recursive call to draw_interval with nonzero parameter makes exactly one direct call to draw_line.

Some intuition may be gained by examining the source code and the recursion trace. We know that a call to draw_interval(c) for $c > 0$ spawns two calls to draw_interval(c−1) and a single call to draw_line. We will rely on this intuition to prove the following claim.

Proposition 4.1: *For $c \geq 0$, a call to draw_interval(c) results in precisely $2^c - 1$ lines of output.*

Justification: We provide a formal proof of this claim by ***induction*** (see Section 3.4.3). In fact, induction is a natural mathematical technique for proving the correctness and efficiency of a recursive process. In the case of the ruler, we note that an application of draw_interval(0) generates no output, and that $2^0 - 1 = 1 - 1 = 0$. This serves as a base case for our claim.

More generally, the number of lines printed by draw_interval(c) is one more than twice the number generated by a call to draw_interval(c−1), as one center line is printed between two such recursive calls. By induction, we have that the number of lines is thus $1 + 2 \cdot (2^{c-1} - 1) = 1 + 2^c - 2 = 2^c - 1$. ■

This proof is indicative of a more mathematically rigorous tool, known as a ***recurrence equation*** that can be used to analyze the running time of a recursive algorithm. That technique is discussed in Section 12.2.4, in the context of recursive sorting algorithms.

Performing a Binary Search

Considering the running time of the binary search algorithm, as presented in Section 4.1.3, we observe that a constant number of primitive operations are executed at each recursive call of method of a binary search. Hence, the running time is proportional to the number of recursive calls performed. We will show that at most $\lfloor \log n \rfloor + 1$ recursive calls are made during a binary search of a sequence having n elements, leading to the following claim.

Proposition 4.2: *The binary search algorithm runs in $O(\log n)$ time for a sorted sequence with n elements.*

Justification: To prove this claim, a crucial fact is that with each recursive call the number of candidate entries still to be searched is given by the value

$$\mathsf{high} - \mathsf{low} + 1.$$

Moreover, the number of remaining candidates is reduced by at least one half with each recursive call. Specifically, from the definition of mid, the number of remaining candidates is either

$$(\mathsf{mid} - 1) - \mathsf{low} + 1 = \left\lfloor \frac{\mathsf{low} + \mathsf{high}}{2} \right\rfloor - \mathsf{low} \leq \frac{\mathsf{high} - \mathsf{low} + 1}{2}$$

or

$$\mathsf{high} - (\mathsf{mid} + 1) + 1 = \mathsf{high} - \left\lfloor \frac{\mathsf{low} + \mathsf{high}}{2} \right\rfloor \leq \frac{\mathsf{high} - \mathsf{low} + 1}{2}.$$

Initially, the number of candidates is n; after the first call in a binary search, it is at most $n/2$; after the second call, it is at most $n/4$; and so on. In general, after the jth call in a binary search, the number of candidate entries remaining is at most $n/2^j$. In the worst case (an unsuccessful search), the recursive calls stop when there are no more candidate entries. Hence, the maximum number of recursive calls performed, is the smallest integer r such that

$$\frac{n}{2^r} < 1.$$

In other words (recalling that we omit a logarithm's base when it is 2), $r > \log n$. Thus, we have

$$r = \lfloor \log n \rfloor + 1,$$

which implies that binary search runs in $O(\log n)$ time. ■

Computing Disk Space Usage

Our final recursive algorithm from Section 4.1 was that for computing the overall disk space usage in a specified portion of a file system. To characterize the "problem size" for our analysis, we let n denote the number of file-system entries in the portion of the file system that is considered. (For example, the file system portrayed in Figure 4.6 has $n = 19$ entries.)

To characterize the cumulative time spent for an initial call to the disk_usage function, we must analyze the total number of recursive invocations that are made, as well as the number of operations that are executed within those invocations.

We begin by showing that there are precisely n recursive invocations of the function, in particular, one for each entry in the relevant portion of the file system. Intuitively, this is because a call to disk_usage for a particular entry e of the file system is only made from within the for loop of Code Fragment 4.5 when processing the entry for the unique directory that contains e, and that entry will only be explored once.

To formalize this argument, we can define the ***nesting level*** of each entry such that the entry on which we begin has nesting level 0, entries stored directly within it have nesting level 1, entries stored within those entries have nesting level 2, and so on. We can prove by induction that there is exactly one recursive invocation of disk_usage upon each entry at nesting level k. As a base case, when $k = 0$, the only recursive invocation made is the initial one. As the inductive step, once we know there is exactly one recursive invocation for each entry at nesting level k, we can claim that there is exactly one invocation for each entry e at nesting level k, made within the for loop for the entry at level k that contains e.

Having established that there is one recursive call for each entry of the file system, we return to the question of the overall computation time for the algorithm. It would be great if we could argue that we spend $O(1)$ time in any single invocation of the function, but that is not the case. While there are a constant number of steps reflect in the call to os.path.getsize to compute the disk usage directly at that entry, when the entry is a directory, the body of the disk_usage function includes a for loop that iterates over all entries that are contained within that directory. In the worst case, it is possible that one entry includes $n - 1$ others.

Based on this reasoning, we could conclude that there are $O(n)$ recursive calls, each of which runs in $O(n)$ time, leading to an overall running time that is $O(n^2)$. While this upper bound is technically true, it is not a tight upper bound. Remarkably, we can prove the stronger bound that the recursive algorithm for disk_usage completes in $O(n)$ time! The weaker bound was pessimistic because it assumed a worst-case number of entries for each directory. While it is possible that some directories contain a number of entries proportional to n, they cannot all contain that many. To prove the stronger claim, we choose to consider the *overall* number of iterations of the for loop across all recursive calls. We claim there are precisely $n - 1$ such iteration of that loop overall. We base this claim on the fact that each iteration of that loop makes a recursive call to disk_usage, and yet we have already concluded that there are a total of n calls to disk_usage (including the original call). We therefore conclude that there are $O(n)$ recursive calls, each of which uses $O(1)$ time outside the loop, and that the *overall* number of operations due to the loop is $O(n)$. Summing all of these bounds, the overall number of operations is $O(n)$.

The argument we have made is more advanced than with the earlier examples of recursion. The idea that we can sometimes get a tighter bound on a series of operations by considering the cumulative effect, rather than assuming that each achieves a worst case is a technique called ***amortization***; we will see a further example of such analysis in Section 5.3. Furthermore, a file system is an implicit example of a data structure known as a ***tree***, and our disk usage algorithm is really a manifestation of a more general algorithm known as a ***tree traversal***. Trees will be the focus of Chapter 8, and our argument about the $O(n)$ running time of the disk usage algorithm will be generalized for tree traversals in Section 8.4.

4.3 Recursion Run Amok

Although recursion is a very powerful tool, it can easily be misused in various ways. In this section, we examine several problems in which a poorly implemented recursion causes drastic inefficiency, and we discuss some strategies for recognizing and avoid such pitfalls.

We begin by revisiting the ***element uniqueness problem***, defined on page 135 of Section 3.3.3. We can use the following recursive formulation to determine if all n elements of a sequence are unique. As a base case, when $n = 1$, the elements are trivially unique. For $n \geq 2$, the elements are unique if and only if the first $n-1$ elements are unique, the last $n-1$ items are unique, and the first and last elements are different (as that is the only pair that was not already checked as a subcase). A recursive implementation based on this idea is given in Code Fragment 4.6, named unique3 (to differentiate it from unique1 and unique2 from Chapter 3).

```
1  def unique3(S, start, stop):
2    """Return True if there are no duplicate elements in slice S[start:stop]."""
3    if stop - start <= 1: return True                 # at most one item
4    elif not unique(S, start, stop-1): return False   # first part has duplicate
5    elif not unique(S, start+1, stop): return False   # second part has duplicate
6    else: return S[start] != S[stop-1]                # do first and last differ?
```

Code Fragment 4.6: Recursive unique3 for testing element uniqueness.

Unfortunately, this is a terribly inefficient use of recursion. The nonrecursive part of each call uses $O(1)$ time, so the overall running time will be proportional to the total number of recursive invocations. To analyze the problem, we let n denote the number of entries under consideration, that is, let n= stop − start.

If $n = 1$, then the running time of unique3 is $O(1)$, since there are no recursive calls for this case. In the general case, the important observation is that a single call to unique3 for a problem of size n may result in two recursive calls on problems of size $n-1$. Those two calls with size $n-1$ could in turn result in four calls (two each) with a range of size $n-2$, and thus eight calls with size $n-3$ and so on. Thus, in the worst case, the total number of function calls is given by the geometric summation

$$1 + 2 + 4 + \cdots + 2^{n-1},$$

which is equal to $2^n - 1$ by Proposition 3.5. Thus, the running time of function unique3 is $O(2^n)$. This is an incredibly inefficient function for solving the element uniqueness problem. Its inefficiency comes not from the fact that it uses recursion—it comes from the fact that it uses recursion poorly, which is something we address in Exercise C-4.11.

An Inefficient Recursion for Computing Fibonacci Numbers

In Section 1.8, we introduced a process for generating the Fibonacci numbers, which can be defined recursively as follows:

$$\begin{aligned} F_0 &= 0 \\ F_1 &= 1 \\ F_n &= F_{n-2} + F_{n-1} \quad \text{for } n > 1. \end{aligned}$$

Ironically, a direct implementation based on this definition results in the function bad_fibonacci shown in Code Fragment 4.7, which computes the sequence of Fibonacci numbers by making two recursive calls in each non-base case.

```
1  def bad_fibonacci(n):
2    """Return the nth Fibonacci number."""
3    if n <= 1:
4      return n
5    else:
6      return bad_fibonacci(n-2) + bad_fibonacci(n-1)
```

Code Fragment 4.7: Computing the nth Fibonacci number using binary recursion.

Unfortunately, such a direct implementation of the Fibonacci formula results in a terribly inefficient function. Computing the nth Fibonacci number in this way requires an exponential number of calls to the function. Specifically, let c_n denote the number of calls performed in the execution of bad_fibonacci(n). Then, we have the following values for the c_n's:

$$\begin{aligned} c_0 &= 1 \\ c_1 &= 1 \\ c_2 &= 1 + c_0 + c_1 = 1 + 1 + 1 = 3 \\ c_3 &= 1 + c_1 + c_2 = 1 + 1 + 3 = 5 \\ c_4 &= 1 + c_2 + c_3 = 1 + 3 + 5 = 9 \\ c_5 &= 1 + c_3 + c_4 = 1 + 5 + 9 = 15 \\ c_6 &= 1 + c_4 + c_5 = 1 + 9 + 15 = 25 \\ c_7 &= 1 + c_5 + c_6 = 1 + 15 + 25 = 41 \\ c_8 &= 1 + c_6 + c_7 = 1 + 25 + 41 = 67 \end{aligned}$$

If we follow the pattern forward, we see that the number of calls more than doubles for each two consecutive indices. That is, c_4 is more than twice c_2, c_5 is more than twice c_3, c_6 is more than twice c_4, and so on. Thus, $c_n > 2^{n/2}$, which means that bad_fibonacci(n) makes a number of calls that is exponential in n.

An Efficient Recursion for Computing Fibonacci Numbers

We were tempted into using the bad recursion formulation because of the way the nth Fibonacci number, F_n, depends on the two previous values, F_{n-2} and F_{n-1}. But notice that after computing F_{n-2}, the call to compute F_{n-1} requires its own recursive call to compute F_{n-2}, as it does not have knowledge of the value of F_{n-2} that was computed at the earlier level of recursion. That is duplicative work. Worse yet, both of those calls will need to (re)compute the value of F_{n-3}, as will the computation of F_{n-1}. This snowballing effect is what leads to the exponential running time of bad_recursion.

We can compute F_n much more efficiently using a recursion in which each invocation makes only one recursive call. To do so, we need to redefine the expectations of the function. Rather than having the function return a single value, which is the nth Fibonacci number, we define a recursive function that returns a pair of consecutive Fibonacci numbers (F_n, F_{n-1}), using the convention $F_{-1} = 0$. Although it seems to be a greater burden to report two consecutive Fibonacci numbers instead of one, passing this extra information from one level of the recursion to the next makes it much easier to continue the process. (It allows us to avoid having to recompute the second value that was already known within the recursion.) An implementation based on this strategy is given in Code Fragment 4.8.

```
1  def good_fibonacci(n):
2    """Return pair of Fibonacci numbers, F(n) and F(n-1)."""
3    if n <= 1:
4      return (n,0)
5    else:
6      (a, b) = good_fibonacci(n-1)
7      return (a+b, a)
```

Code Fragment 4.8: Computing the nth Fibonacci number using linear recursion.

In terms of efficiency, the difference between the bad recursion and the good recursion for this problem is like night and day. The bad_fibonacci function uses exponential time. We claim that the execution of function good_fibonacci(n) takes $O(n)$ time. Each recursive call to good_fibonacci decreases the argument n by 1; therefore, a recursion trace includes a series of n function calls. Because the nonrecursive work for each call uses constant time, the overall computation executes in $O(n)$ time.

4.3.1 Maximum Recursive Depth in Python

Another danger in the misuse of recursion is known as ***infinite recursion***. If each recursive call makes another recursive call, without ever reaching a base case, then we have an infinite series of such calls. This is a fatal error. An infinite recursion can quickly swamp computing resources, not only due to rapid use of the CPU, but because each successive call creates an activation record requiring additional memory. A blatant example of an ill-formed recursion is the following:

```
def fib(n):
  return fib(n)                    # fib(n) equals fib(n)
```

However, there are far more subtle errors that can lead to an infinite recursion. Revisiting our implementation of binary search in Code Fragment 4.3, in the final case (line 17) we make a recursive call on the right portion of the sequence, in particular going from index mid+1 to high. Had that line instead been written as

```
    return binary_search(data, target, mid, high)      # note the use of mid
```

this could result in an infinite recursion. In particular, when searching a range of two elements, it becomes possible to make a recursive call on the identical range.

A programmer should ensure that each recursive call is in some way progressing toward a base case (for example, by having a parameter value that decreases with each call). However, to combat against infinite recursions, the designers of Python made an intentional decision to limit the overall number of function activations that can be simultaneously active. The precise value of this limit depends upon the Python distribution, but a typical default value is 1000. If this limit is reached, the Python interpreter raises a RuntimeError with a message, `maximum recursion depth exceeded`.

For many legitimate applications of recursion, a limit of 1000 nested function calls suffices. For example, our binary_search function (Section 4.1.3) has $O(\log n)$ recursive depth, and so for the default recursive limit to be reached, there would need to be 2^{1000} elements (far, far more than the estimated number of atoms in the universe). However, in the next section we discuss several algorithms that have recursive depth proportional to n. Python's artificial limit on the recursive depth could disrupt such otherwise legitimate computations.

Fortunately, the Python interpreter can be dynamically reconfigured to change the default recursive limit. This is done through use of a module named sys, which supports a getrecursionlimit function and a setrecursionlimit. Sample usage of those functions is demonstrated as follows:

```
import sys
old = sys.getrecursionlimit( )       # perhaps 1000 is typical
sys.setrecursionlimit(1000000)       # change to allow 1 million nested calls
```

4.4 Further Examples of Recursion

In the remainder of this chapter, we provide additional examples of the use of recursion. We organize our presentation by considering the maximum number of recursive calls that may be started from within the body of a single activation.

- If a recursive call starts at most one other, we call this a ***linear recursion***.
- If a recursive call may start two others, we call this a ***binary recursion***.
- If a recursive call may start three or more others, this is ***multiple recursion***.

4.4.1 Linear Recursion

If a recursive function is designed so that each invocation of the body makes at most one new recursive call, this is know as ***linear recursion***. Of the recursions we have seen so far, the implementation of the factorial function (Section 4.1.1) and the good_fibonacci function (Section 4.3) are clear examples of linear recursion. More interestingly, the binary search algorithm (Section 4.1.3) is also an example of *linear recursion*, despite the "binary" terminology in the name. The code for binary search (Code Fragment 4.3) includes a case analysis with two branches that lead to recursive calls, but only one of those calls can be reached during a particular execution of the body.

A consequence of the definition of linear recursion is that any recursion trace will appear as a single sequence of calls, as we originally portrayed for the factorial function in Figure 4.1 of Section 4.1.1. Note that the *linear recursion* terminology reflects the structure of the recursion trace, not the asymptotic analysis of the running time; for example, we have seen that binary search runs in $O(\log n)$ time.

Summing the Elements of a Sequence Recursively

Linear recursion can be a useful tool for processing a data sequence, such as a Python list. Suppose, for example, that we want to compute the sum of a sequence, S, of n integers. We can solve this summation problem using linear recursion by observing that the sum of all n integers in S is trivially 0, if $n = 0$, and otherwise that it is the sum of the first $n-1$ integers in S plus the last element in S. (See Figure 4.9.)

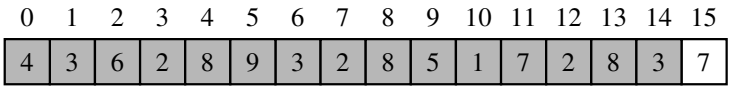


Figure 4.9: Computing the sum of a sequence recursively, by adding the last number to the sum of the first $n-1$.

A recursive algorithm for computing the sum of a sequence of numbers based on this intuition is implemented in Code Fragment 4.9.

```
1  def linear_sum(S, n):
2    """Return the sum of the first n numbers of sequence S."""
3    if n == 0:
4      return 0
5    else:
6      return linear_sum(S, n−1) + S[n−1]
```

Code Fragment 4.9: Summing the elements of a sequence using linear recursion.

A recursion trace of the linear_sum function for a small example is given in Figure 4.10. For an input of size n, the linear_sum algorithm makes $n+1$ function calls. Hence, it will take $O(n)$ time, because it spends a constant amount of time performing the nonrecursive part of each call. Moreover, we can also see that the memory space used by the algorithm (in addition to the sequence S) is also $O(n)$, as we use a constant amount of memory space for each of the $n+1$ activation records in the trace at the time we make the final recursive call (with $n = 0$).

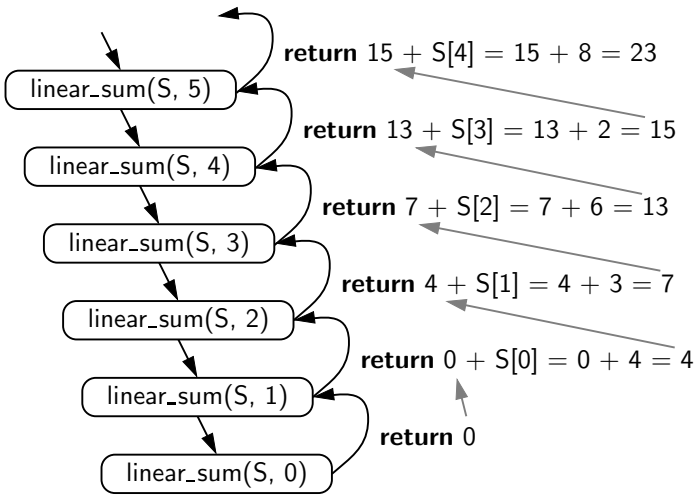


Figure 4.10: Recursion trace for an execution of linear_sum(S, 5) with input parameter S = [4, 3, 6, 2, 8].

Reversing a Sequence with Recursion

Next, let us consider the problem of reversing the n elements of a sequence, S, so that the first element becomes the last, the second element becomes second to the last, and so on. We can solve this problem using linear recursion, by observing that the reversal of a sequence can be achieved by swapping the first and last elements and then recursively reversing the remaining elements. We present an implementation of this algorithm in Code Fragment 4.10, using the convention that the first time we call this algorithm we do so as reverse(S, 0, len(S)).

```
1  def reverse(S, start, stop):
2    """Reverse elements in implicit slice S[start:stop]."""
3    if start < stop - 1:                    # if at least 2 elements:
4      S[start], S[stop-1] = S[stop-1], S[start]    # swap first and last
5      reverse(S, start+1, stop-1)                  # recur on rest
```

Code Fragment 4.10: Reversing the elements of a sequence using linear recursion.

Note that there are two implicit base case scenarios: When start == stop, the implicit range is empty, and when start == stop−1, the implicit range has only one element. In either of these cases, there is no need for action, as a sequence with zero elements or one element is trivially equal to its reversal. When otherwise invoking recursion, we are guaranteed to make progress towards a base case, as the difference, stop−start, decreases by two with each call (see Figure 4.11). If n is even, we will eventually reach the start == stop case, and if n is odd, we will eventually reach the start == stop − 1 case.

The above argument implies that the recursive algorithm of Code Fragment 4.10 is guaranteed to terminate after a total of $1 + \lfloor \frac{n}{2} \rfloor$ recursive calls. Since each call involves a constant amount of work, the entire process runs in $O(n)$ time.

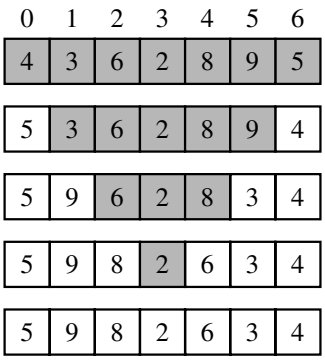


Figure 4.11: A trace of the recursion for reversing a sequence. The shaded portion has yet to be reversed.

Recursive Algorithms for Computing Powers

As another interesting example of the use of linear recursion, we consider the problem of raising a number x to an arbitrary nonnegative integer, n. That is, we wish to compute the ***power function***, defined as $power(x,n) = x^n$. (We use the name "power" for this discussion, to differentiate from the built-in function pow that provides such functionality.) We will consider two different recursive formulations for the problem that lead to algorithms with very different performance.

A trivial recursive definition follows from the fact that $x^n = x \cdot x^{n-1}$ for $n > 0$.

$$power(x,n) = \begin{cases} 1 & \text{if } n = 0 \\ x \cdot power(x, n-1) & \text{otherwise.} \end{cases}$$

This definition leads to a recursive algorithm shown in Code Fragment 4.11.

```
1  def power(x, n):
2    """Compute the value x**n for integer n."""
3    if n == 0:
4      return 1
5    else:
6      return x * power(x, n-1)
```

Code Fragment 4.11: Computing the power function using trivial recursion.

A recursive call to this version of $power(x,n)$ runs in $O(n)$ time. Its recursion trace has structure very similar to that of the factorial function from Figure 4.1, with the parameter decreasing by one with each call, and constant work performed at each of $n+1$ levels.

However, there is a much faster way to compute the power function using an alternative definition that employs a squaring technique. Let $k = \lfloor \frac{n}{2} \rfloor$ denote the floor of the division (expressed as n // 2 in Python). We consider the expression $\left(x^k\right)^2$. When n is even, $\lfloor \frac{n}{2} \rfloor = \frac{n}{2}$ and therefore $\left(x^k\right)^2 = \left(x^{\frac{n}{2}}\right)^2 = x^n$. When n is odd, $\lfloor \frac{n}{2} \rfloor = \frac{n-1}{2}$ and $\left(x^k\right)^2 = x^{n-1}$, and therefore $x^n = x \cdot \left(x^k\right)^2$, just as $2^{13} = 2 \cdot 2^6 \cdot 2^6$. This analysis leads to the following recursive definition:

$$power(x,n) = \begin{cases} 1 & \text{if } n = 0 \\ x \cdot \left(power\left(x, \lfloor \frac{n}{2} \rfloor\right)\right)^2 & \text{if } n > 0 \text{ is odd} \\ \left(power\left(x, \lfloor \frac{n}{2} \rfloor\right)\right)^2 & \text{if } n > 0 \text{ is even} \end{cases}$$

If we were to implement this recursion making *two* recursive calls to compute $power(x, \lfloor \frac{n}{2} \rfloor) \cdot power(x, \lfloor \frac{n}{2} \rfloor)$, a trace of the recursion would demonstrate $O(n)$ calls. We can perform significantly fewer operations by computing $power(x, \lfloor \frac{n}{2} \rfloor)$ as a partial result, and then multiplying it by itself. An implementation based on this recursive definition is given in Code Fragment 4.12.

```
1  def power(x, n):
2    """Compute the value x**n for integer n."""
3    if n == 0:
4      return 1
5    else:
6      partial = power(x, n // 2)          # rely on truncated division
7      result = partial * partial
8      if n % 2 == 1:                      # if n odd, include extra factor of x
9        result *= x
10     return result
```

Code Fragment 4.12: Computing the power function using repeated squaring.

To illustrate the execution of our improved algorithm, Figure 4.12 provides a recursion trace of the computation power(2, 13).

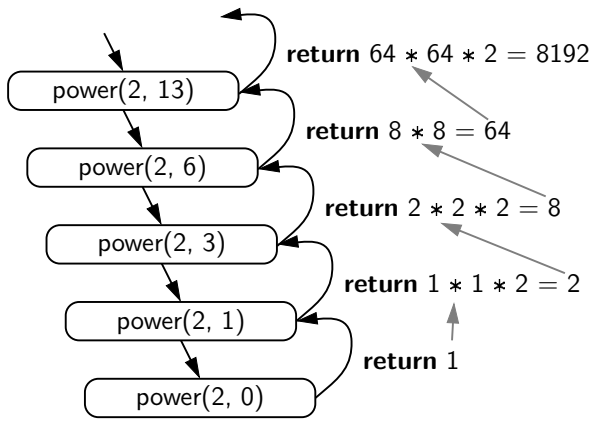


Figure 4.12: Recursion trace for an execution of power(2, 13).

To analyze the running time of the revised algorithm, we observe that the exponent in each recursive call of function power(x,n) is at most half of the preceding exponent. As we saw with the analysis of binary search, the number of times that we can divide n in half before getting to one or less is $O(\log n)$. Therefore, our new formulation of the power function results in $O(\log n)$ recursive calls. Each individual activation of the function uses $O(1)$ operations (excluding the recursive calls), and so the total number of operations for computing power(x,n) is $O(\log n)$. This is a significant improvement over the original $O(n)$-time algorithm.

The improved version also provides significant saving in reducing the memory usage. The first version has a recursive depth of $O(n)$, and therefore $O(n)$ activation records are simultaneous stored in memory. Because the recursive depth of the improved version is $O(\log n)$, its memory usages is $O(\log n)$ as well.

4.4.2 Binary Recursion

When a function makes two recursive calls, we say that it uses ***binary recursion***. We have already seen several examples of binary recursion, most notably when drawing the English ruler (Section 4.1.2), or in the bad_fibonacci function of Section 4.3. As another application of binary recursion, let us revisit the problem of summing the n elements of a sequence, S, of numbers. Computing the sum of one or zero elements is trivial. With two or more elements, we can recursively compute the sum of the first half, and the sum of the second half, and add these sums together. Our implementation of such an algorithm, in Code Fragment 4.13, is initially invoked as binary_sum(A, 0, len(A)).

```
1  def binary_sum(S, start, stop):
2    """Return the sum of the numbers in implicit slice S[start:stop]."""
3    if start >= stop:                          # zero elements in slice
4      return 0
5    elif start == stop-1:                      # one element in slice
6      return S[start]
7    else:                                      # two or more elements in slice
8      mid = (start + stop) // 2
9      return binary_sum(S, start, mid) + binary_sum(S, mid, stop)
```

Code Fragment 4.13: Summing the elements of a sequence using binary recursion.

To analyze algorithm binary_sum, we consider, for simplicity, the case where n is a power of two. Figure 4.13 shows the recursion trace of an execution of binary_sum(0, 8). We label each box with the values of parameters start:stop for that call. The size of the range is divided in half at each recursive call, and so the depth of the recursion is $1+\log_2 n$. Therefore, binary_sum uses $O(\log n)$ amount of additional space, which is a big improvement over the $O(n)$ space used by the linear_sum function of Code Fragment 4.9. However, the running time of binary_sum is $O(n)$, as there are $2n-1$ function calls, each requiring constant time.

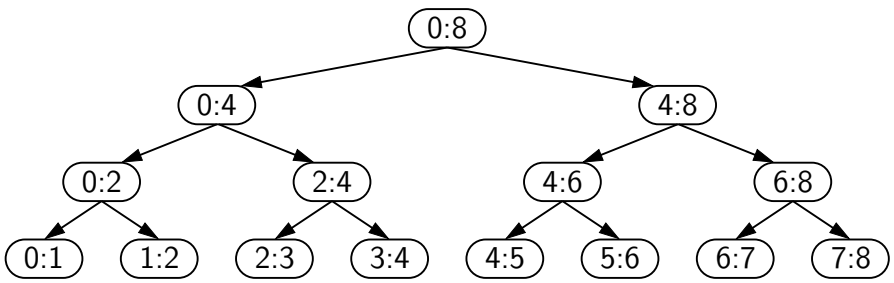


Figure 4.13: Recursion trace for the execution of binary_sum(0, 8).

4.4.3 Multiple Recursion

Generalizing from binary recursion, we define ***multiple recursion*** as a process in which a function may make more than two recursive calls. Our recursion for analyzing the disk space usage of a file system (see Section 4.1.4) is an example of multiple recursion, because the number of recursive calls made during one invocation was equal to the number of entries within a given directory of the file system.

Another common application of multiple recursion is when we want to enumerate various configurations in order to solve a combinatorial puzzle. For example, the following are all instances of what are known as ***summation puzzles***:

$$\begin{aligned} pot + pan &= bib \\ dog + cat &= pig \\ boy + girl &= baby \end{aligned}$$

To solve such a puzzle, we need to assign a unique digit (that is, $0, 1, \ldots, 9$) to each letter in the equation, in order to make the equation true. Typically, we solve such a puzzle by using our human observations of the particular puzzle we are trying to solve to eliminate configurations (that is, possible partial assignments of digits to letters) until we can work though the feasible configurations left, testing for the correctness of each one.

If the number of possible configurations is not too large, however, we can use a computer to simply enumerate all the possibilities and test each one, without employing any human observations. In addition, such an algorithm can use multiple recursion to work through the configurations in a systematic way. We show pseudocode for such an algorithm in Code Fragment 4.14. To keep the description general enough to be used with other puzzles, the algorithm enumerates and tests all k-length sequences without repetitions of the elements of a given universe U. We build the sequences of k elements by the following steps:

1. Recursively generating the sequences of $k-1$ elements
2. Appending to each such sequence an element not already contained in it.

Throughout the execution of the algorithm, we use a set U to keep track of the elements not contained in the current sequence, so that an element e has not been used yet if and only if e is in U.

Another way to look at the algorithm of Code Fragment 4.14 is that it enumerates every possible size-k ordered subset of U, and tests each subset for being a possible solution to our puzzle.

For summation puzzles, $U = \{0,1,2,3,4,5,6,7,8,9\}$ and each position in the sequence corresponds to a given letter. For example, the first position could stand for b, the second for o, the third for y, and so on.

```
Algorithm PuzzleSolve(k,S,U):
    Input: An integer k, sequence S, and set U
    Output: An enumeration of all k-length extensions to S using elements in U
      without repetitions
    for each e in U do
      Add e to the end of S
      Remove e from U                                   {e is now being used}
      if k == 1 then
        Test whether S is a configuration that solves the puzzle
        if S solves the puzzle then
          return "Solution found: " S
      else
        PuzzleSolve(k−1,S,U)                            {a recursive call}
      Remove e from the end of S
      Add e back to U                                   {e is now considered as unused}
```

Code Fragment 4.14: Solving a combinatorial puzzle by enumerating and testing all possible configurations.

In Figure 4.14, we show a recursion trace of a call to PuzzleSolve$(3,S,U)$, where S is empty and $U = \{a,b,c\}$. During the execution, all the permutations of the three characters are generated and tested. Note that the initial call makes three recursive calls, each of which in turn makes two more. If we had executed PuzzleSolve$(3,S,U)$ on a set U consisting of four elements, the initial call would have made four recursive calls, each of which would have a trace looking like the one in Figure 4.14.

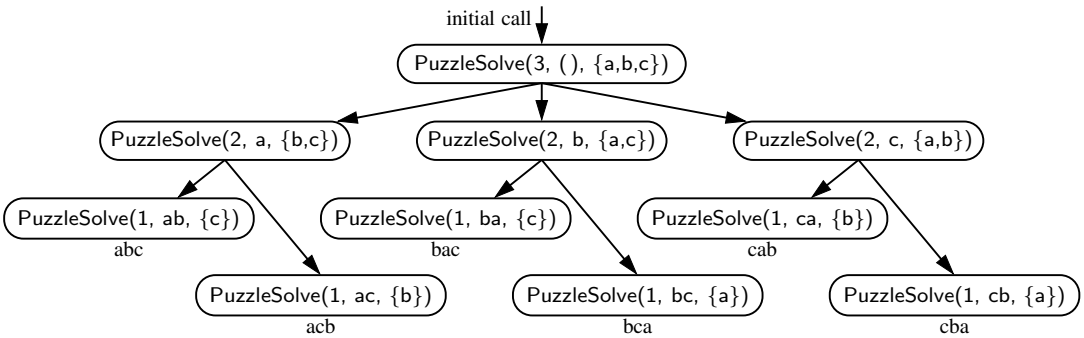


Figure 4.14: Recursion trace for an execution of PuzzleSolve$(3,S,U)$, where S is empty and $U = \{a,b,c\}$. This execution generates and tests all permutations of a, b, and c. We show the permutations generated directly below their respective boxes.

4.5 Designing Recursive Algorithms

In general, an algorithm that uses recursion typically has the following form:

- ***Test for base cases.*** We begin by testing for a set of base cases (there should be at least one). These base cases should be defined so that every possible chain of recursive calls will eventually reach a base case, and the handling of each base case should not use recursion.
- ***Recur.*** If not a base case, we perform one or more recursive calls. This recursive step may involve a test that decides which of several possible recursive calls to make. We should define each possible recursive call so that it makes progress towards a base case.

Parameterizing a Recursion

To design a recursive algorithm for a given problem, it is useful to think of the different ways we might define subproblems that have the same general structure as the original problem. If one has difficulty finding the repetitive structure needed to design a recursive algorithm, it is sometimes useful to work out the problem on a few concrete examples to see how the subproblems should be defined.

A successful recursive design sometimes requires that we redefine the original problem to facilitate similar-looking subproblems. Often, this involved reparameterizing the signature of the function. For example, when performing a binary search in a sequence, a natural function signature for a caller would appear as binary_search(data, target). However, in Section 4.1.3, we defined our function with calling signature binary_search(data, target, low, high), using the additional parameters to demarcate sublists as the recursion proceeds. This change in parameterization is critical for binary search. If we had insisted on the cleaner signature, binary_search(data, target), the only way to invoke a search on half the list would have been to make a new list instance with only those elements to send as the first parameter. However, making a copy of half the list would already take $O(n)$ time, negating the whole benefit of the binary search algorithm.

If we wished to provide a cleaner public interface to an algorithm like binary search, without bothering a user with the extra parameters, a standard technique is to make one function for public use with the cleaner interface, such as binary_search(data, target), and then having its body invoke a nonpublic utility function having the desired recursive parameters.

You will see that we similarly reparameterized the recursion in several other examples of this chapter (e.g., reverse, linear_sum, binary_sum). We saw a different approach to redefining a recursion in our good_fibonacci implementation, by intentionally strengthening the expectation of what is returned (in that case, returning a pair of numbers rather than a single number).

4.6 Eliminating Tail Recursion

The main benefit of a recursive approach to algorithm design is that it allows us to succinctly take advantage of a repetitive structure present in many problems. By making our algorithm description exploit the repetitive structure in a recursive way, we can often avoid complex case analyses and nested loops. This approach can lead to more readable algorithm descriptions, while still being quite efficient.

However, the usefulness of recursion comes at a modest cost. In particular, the Python interpreter must maintain activation records that keep track of the state of each nested call. When computer memory is at a premium, it is useful in some cases to be able to derive nonrecursive algorithms from recursive ones.

In general, we can use the stack data structure, which we will introduce in Section 6.1, to convert a recursive algorithm into a nonrecursive algorithm by managing the nesting of the recursive structure ourselves, rather than relying on the interpreter to do so. Although this only shifts the memory usage from the interpreter to our stack, we may be able to reduce the memory usage by storing only the minimal information necessary.

Even better, some forms of recursion can be eliminated without any use of axillary memory. A notable such form is known as ***tail recursion***. A recursion is a tail recursion if any recursive call that is made from one context is the very last operation in that context, with the return value of the recursive call (if any) immediately returned by the enclosing recursion. By necessity, a tail recursion must be a linear recursion (since there is no way to make a second recursive call if you must immediately return the result of the first).

Of the recursive functions demonstrated in this chapter, the binary_search function of Code Fragment 4.3 and the reverse function of Code Fragment 4.10 are examples of tail recursion. Several others of our linear recursions are almost like tail recursion, but not technically so. For example, our factorial function of Code Fragment 4.1 is *not* a tail recursion. It concludes with the command:

```
return n * factorial(n-1)
```

This is not a tail recursion because an additional multiplication is performed after the recursive call is completed. For similar reasons, the linear_sum function of Code Fragment 4.9 and the good_fibonacci function of Code Fragment 4.7 fail to be tail recursions.

Any tail recursion can be reimplemented nonrecursively by enclosing the body in a loop for repetition, and replacing a recursive call with new parameters by a reassignment of the existing parameters to those values. As a tangible example, our binary_search function can be reimplemented as shown in Code Fragment 4.15. We initialize variables low and high, just prior to our while loop, to represent the full extent of the sequence. Then, during each pass of the loop, we either find

```
1  def binary_search_iterative(data, target):
2    """Return True if target is found in the given Python list."""
3    low = 0
4    high = len(data)-1
5    while low <= high:
6      mid = (low + high) // 2
7      if target == data[mid]:                # found a match
8        return True
9      elif target < data[mid]:
10       high = mid - 1                       # only consider values left of mid
11     else:
12       low = mid + 1                        # only consider values right of mid
13   return False                             # loop ended without success
```

Code Fragment 4.15: A nonrecursive implementation of binary search.

the target, or we narrow the range of the candidate subsequence. Where we made the recursive call binary_search(data, target, low, mid -1) in the original version, we simply replace high = mid $-$ 1 in our new version and then continue to the next iteration of the loop. Our original base case condition of low $>$ high has simply been replaced by the opposite loop condition **while** low $<=$ high. In our new implementation, we return False to designate a failed search if the while loop ends (that is, without having ever returned True from within).

We can similarly develop a nonrecursive implementation (Code Fragment 4.16) of the original recursive reverse method of Code Fragment 4.10.

```
1  def reverse_iterative(S):
2    """Reverse elements in sequence S."""
3    start, stop = 0, len(S)
4    while start < stop - 1:
5      S[start], S[stop-1] = S[stop-1], S[start]      # swap first and last
6      start, stop = start + 1, stop - 1              # narrow the range
```

Code Fragment 4.16: Reversing the elements of a sequence using iteration.

In this new version, we update the values start and stop during each pass of the loop, exiting once we reach the case of having one or less elements in that range.

Many other linear recursions can be expressed quite efficiently with iteration, even if they were not formally tail recursions. For example, there are trivial nonrecursive implementations for computing factorials, summing elements of a sequence, or computing Fibonacci numbers efficiently. In fact, our implementation of a Fibonacci generator, from Section 1.8, produces each subsequent value in $O(1)$ time, and thus takes $O(n)$ time to generate the nth entry in the series.

4.7 Exercises

For help with exercises, please visit the site, www.wiley.com/college/goodrich.

Reinforcement

R-4.1 Describe a recursive algorithm for finding the maximum element in a sequence, S, of n elements. What is your running time and space usage?

R-4.2 Draw the recursion trace for the computation of $power(2,5)$, using the traditional function implemented in Code Fragment 4.11.

R-4.3 Draw the recursion trace for the computation of $power(2,18)$, using the repeated squaring algorithm, as implemented in Code Fragment 4.12.

R-4.4 Draw the recursion trace for the execution of function reverse(S, 0, 5) (Code Fragment 4.10) on S = [4, 3, 6, 2, 6].

R-4.5 Draw the recursion trace for the execution of function PuzzleSolve$(3,S,U)$ (Code Fragment 4.14), where S is empty and $U = \{a,b,c,d\}$.

R-4.6 Describe a recursive function for computing the nth ***Harmonic number***, $H_n = \sum_{i=1}^{n} 1/i$.

R-4.7 Describe a recursive function for converting a string of digits into the integer it represents. For example, `'13531'` represents the integer 13,531.

R-4.8 Isabel has an interesting way of summing up the values in a sequence A of n integers, where n is a power of two. She creates a new sequence B of half the size of A and sets $B[i] = A[2i] + A[2i+1]$, for $i = 0,1,\ldots,(n/2)-1$. If B has size 1, then she outputs $B[0]$. Otherwise, she replaces A with B, and repeats the process. What is the running time of her algorithm?

Creativity

C-4.9 Write a short recursive Python function that finds the minimum and maximum values in a sequence without using any loops.

C-4.10 Describe a recursive algorithm to compute the integer part of the base-two logarithm of n using only addition and integer division.

C-4.11 Describe an efficient recursive function for solving the element uniqueness problem, which runs in time that is at most $O(n^2)$ in the worst case without using sorting.

C-4.12 Give a recursive algorithm to compute the product of two positive integers, m and n, using only addition and subtraction.

C-4.13 In Section 4.2 we prove by induction that the number of *lines* printed by a call to draw_interval(c) is $2^c - 1$. Another interesting question is how many *dashes* are printed during that process. Prove by induction that the number of dashes printed by draw_interval(c) is $2^{c+1} - c - 2$.

C-4.14 In the ***Towers of Hanoi*** puzzle, we are given a platform with three pegs, a, b, and c, sticking out of it. On peg a is a stack of n disks, each larger than the next, so that the smallest is on the top and the largest is on the bottom. The puzzle is to move all the disks from peg a to peg c, moving one disk at a time, so that we never place a larger disk on top of a smaller one. See Figure 4.15 for an example of the case $n = 4$. Describe a recursive algorithm for solving the Towers of Hanoi puzzle for arbitrary n. (Hint: Consider first the subproblem of moving all but the nth disk from peg a to another peg using the third as "temporary storage.")

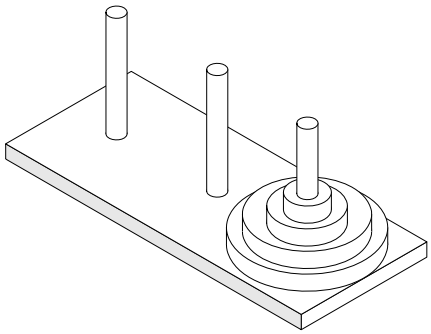


Figure 4.15: An illustration of the Towers of Hanoi puzzle.

C-4.15 Write a recursive function that will output all the subsets of a set of n elements (without repeating any subsets).

C-4.16 Write a short recursive Python function that takes a character string s and outputs its reverse. For example, the reverse of `'pots&pans'` would be `'snap&stop'`.

C-4.17 Write a short recursive Python function that determines if a string s is a palindrome, that is, it is equal to its reverse. For example, `'racecar'` and `'gohangasalamiimalasagnahog'` are palindromes.

C-4.18 Use recursion to write a Python function for determining if a string s has more vowels than consonants.

C-4.19 Write a short recursive Python function that rearranges a sequence of integer values so that all the even values appear before all the odd values.

C-4.20 Given an unsorted sequence, S, of integers and an integer k, describe a recursive algorithm for rearranging the elements in S so that all elements less than or equal to k come before any elements larger than k. What is the running time of your algorithm on a sequence of n values?

C-4.21 Suppose you are given an n-element sequence, S, containing distinct integers that are listed in increasing order. Given a number k, describe a recursive algorithm to find two integers in S that sum to k, if such a pair exists. What is the running time of your algorithm?

C-4.22 Develop a nonrecursive implementation of the version of power from Code Fragment 4.12 that uses repeated squaring.

Projects

P-4.23 Implement a recursive function with signature find(path, filename) that reports all entries of the file system rooted at the given path having the given file name.

P-4.24 Write a program for solving summation puzzles by enumerating and testing all possible configurations. Using your program, solve the three puzzles given in Section 4.4.3.

P-4.25 Provide a nonrecursive implementation of the draw_interval function for the English ruler project of Section 4.1.2. There should be precisely $2^c - 1$ lines of output if c represents the length of the center tick. If incrementing a counter from 0 to $2^c - 2$, the number of dashes for each tick line should be exactly one more than the number of consecutive 1's at the end of the binary representation of the counter.

P-4.26 Write a program that can solve instances of the Tower of Hanoi problem (from Exercise C-4.14).

P-4.27 Python's os module provides a function with signature walk(path) that is a generator yielding the tuple (dirpath, dirnames, filenames) for each subdirectory of the directory identified by string path, such that string dirpath is the full path to the subdirectory, dirnames is a list of the names of the subdirectories within dirpath, and filenames is a list of the names of non-directory entries of dirpath. For example, when visiting the cs016 subdirectory of the file system shown in Figure 4.6, the walk would yield ('/user/rt/courses/cs016', ['homeworks', 'programs'], ['grades']). Give your own implementation of such a walk function.

Chapter Notes

The use of recursion in programs belongs to the folkore of computer science (for example, see the article of Dijkstra [36]). It is also at the heart of functional programming languages (for example, see the classic book by Abelson, Sussman, and Sussman [1]). Interestingly, binary search was first published in 1946, but was not published in a fully correct form until 1962. For further discussions on lessons learned, please see papers by Bentley [14] and Lesuisse [68].